

DAY 24

Introduction to Machine Learning with Python

Prepared for Muhammad Ahmad

60-Day Data Science Learning Journey

Python • Scikit-learn • Linear Regression • Model Evaluation

Learning Objectives

- Understand Machine Learning and its main types.
- Understand features, targets, training data, and test data.
- Build a supervised regression model with Scikit-learn.
- Evaluate predictions with MAE, MSE, and R^2 .
- Understand model limitations and responsible interpretation.

1. What is Machine Learning?

Machine Learning is a branch of Artificial Intelligence in which algorithms learn patterns from data and use those patterns to make predictions or decisions. Instead of manually writing every rule, we provide examples and an algorithm learns model parameters.

Traditional: Input + rules → output

Machine Learning: Examples + algorithm → trained model → prediction

Example: use historical study hours and exam marks to estimate marks for a new example. This is an educational demonstration, not a system for real academic decisions.

2. Main Types

Type	Core idea	Example
Supervised	Learns from labeled examples	Predict marks or classify spam
Unsupervised	Finds structure without target labels	Customer clustering
Reinforcement	Learns through rewards and penalties	Game-playing agent

Day 24 focus: supervised learning, specifically regression, where the target is numeric.

3. Machine Learning Vocabulary

Term	Meaning	Example
Dataset	Collection of observations	Student records
Feature (X)	Input used for prediction	Study hours
Target (y)	Value to predict	Exam marks
Training data	Examples used to fit model	80% of records
Test data	Held-out evaluation examples	20% of records
Model	Learned relationship	Regression model
Prediction	Model output	Estimated marks

4. First Project: Student Marks Prediction

We use a small illustrative dataset where study hours are the feature and exam marks are the target.

Install

```
pip install numpy pandas matplotlib scikit-learn
```

Complete code

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

data = {
    "study_hours": [1,2,3,4,5,6,7,8,2,4,6,8,3,5,7,1,9,10,9,10],
    "marks": [35,40,50,55,65,70,80,85,42,58,72,88,48,63,78,32,90,96,93,98]
}

df = pd.DataFrame(data)

X = df[["study_hours"]]
y = df["marks"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Actual marks:", y_test.tolist())
print("Predicted marks:", y_pred.round(2).tolist())
print("MAE:", round(mae, 2))
print("MSE:", round(mse, 2))
print("R-squared:", round(r2, 3))

new_student = pd.DataFrame({"study_hours": [6.5]})
predicted_marks = model.predict(new_student)

print("Predicted marks for 6.5 study hours:",
      round(predicted_marks[0], 2))

plt.scatter(df["study_hours"], df["marks"], label="Sample data")
plt.plot(df["study_hours"],
         model.predict(df[["study_hours"]]),
         label="Learned regression line")
plt.xlabel("Study Hours")
plt.ylabel("Exam Marks")
plt.title("Student Marks Prediction")
plt.legend()
plt.tight_layout()
plt.show()
```

5. Line-by-Line Explanation

Imports

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

- Pandas manages the data table.
- Matplotlib creates the chart.

- `train_test_split` creates training and testing subsets.
- `LinearRegression` provides the regression algorithm.
- The three metrics evaluate prediction performance.

Feature and target

```
X = df[["study_hours"]]
y = df["marks"]
```

X contains the input feature. The double brackets keep X two-dimensional. y contains the value to predict.

Train/test split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

- 80% is used for training and 20% for testing in this example.
- The test set is not used to fit the model.
- `random_state=42` makes the split reproducible.
- 80/20 is a common starting point, not a universal rule.

Train the model

```
model = LinearRegression()
model.fit(X_train, y_train)
```

The model learns a straight-line relationship. The basic equation is $\hat{y} = b_0 + b_1X$, where $\hat{y}$ is the prediction, b_0 is the intercept, and b_1 is the learned coefficient.

Prediction and metrics

```
y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

- MAE is average absolute error in target units. Lower is generally better.
- MSE averages squared errors and penalizes larger errors more strongly. Lower is better.
- R^2 compares the model with a mean-prediction baseline. It should not be used alone.

New prediction

```
new_student = pd.DataFrame({"study_hours": [6.5]})
predicted_marks = model.predict(new_student)
```

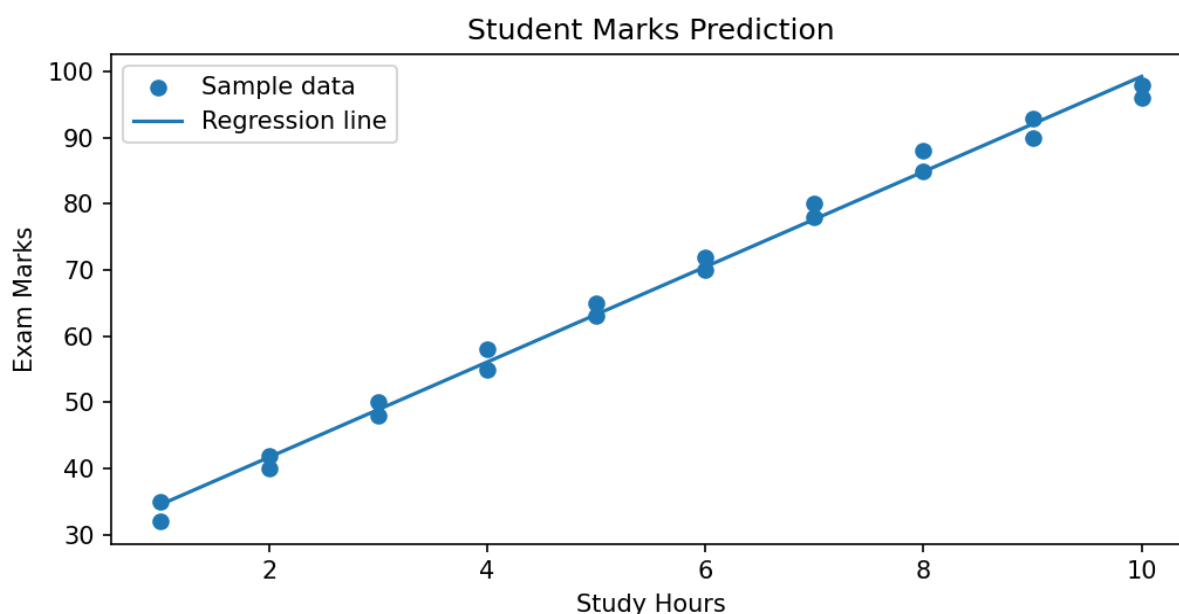
The model estimates a mark for 6.5 study hours. It is a prediction, not a guarantee.

6. Model Evaluation

Metric	Meaning	Preferred
MAE	Average absolute prediction error	Lower
MSE	Average squared prediction error	Lower
R^2	Performance relative to a mean baseline	Often higher, context matters

Important: A strong score on a tiny artificial dataset does not mean a model is ready for real-world deployment.

7. Visualization



The points are sample observations and the line is the fitted relationship. Real workflows should also inspect residuals, outliers, assumptions, leakage, and representative unseen data.

8. Practice Tasks with Solutions

1. Feature vs target

Solution: A feature is an input used by the model. A target is the value the model predicts.

2. Why separate test data?

Solution: To evaluate generalization on examples that were not used to fit the model.

3. What does `random_state=42` do?

Solution: It makes the random split reproducible. 42 is conventional, not mathematically special.

4. What does lower MAE mean?

Solution: Predictions are closer to actual values on average, measured in target units.

5. Adapt the project for house prices

Solution: Use house size or other appropriate features as X and price as y. Clean data, split properly, train, evaluate, and validate on representative data.

6. Why not use this for real academic decisions?

Solution: The dataset is tiny, artificial, and uses only one feature. It has not been validated for real students.

7. What if `test_size` becomes 0.4?

Solution: More data is reserved for testing and less for training. With small data, this can affect both learning and evaluation stability.

8. Why can regression predictions be decimals?

Solution: Regression predicts continuous numeric values, even if a final grading system rounds them.

9. Is correlation enough to build a model?

Solution: No. Modeling also requires data quality, suitable features, validation, assumptions, and appropriate evaluation.

10. What is overfitting?

Solution: Learning training-specific noise so closely that performance on unseen data becomes poor.

9. Mini Project Challenge

Extend the Student Marks Prediction project with at least three features, such as study hours, attendance, and sleep hours. Use responsible simulated data or a suitable real dataset.

- Inspect and clean the data with Pandas.
- Separate features and target.
- Split before fitting.
- Train a regression model.
- Calculate MAE, MSE, and R^2 .
- Plot actual versus predicted values.
- Explain limitations and avoid claiming that predictions prove causation.

10. Machine Learning Workflow

- Define the problem and target.
- Collect appropriate data.
- Explore and clean the data.
- Select or engineer features.
- Split into training and testing data.
- Train the model.
- Evaluate on unseen data.
- Analyze errors and limitations.
- Improve and validate again.
- Only then consider deployment and monitoring.

11. Quick Revision

- Machine Learning learns patterns from examples.
- Supervised learning uses labeled examples.
- Regression predicts numeric values.
- Features are inputs and the target is the prediction goal.
- Training data fit the model and test data evaluate generalization.
- Linear regression learns $y = b_0 + b_1X$.
- MAE and MSE measure prediction error.
- R^2 is useful but should not be used alone.
- Data quality and validation are as important as the algorithm.

End of Day 24 • Prepared for Muhammad Ahmad • 60-Day Data Science Learning Journey